

Anurag Rana, Tithi Jana, Sandeep Jammula, Nishith Guda

CS 6360.002

Group 5

Phase 4

## **Project Description**

Jungle Library is a library for all ages. Jungle Library seeks to implement a centralized relational database system to streamline and structure its operational processes. The proposed database design will incorporate the following core entities: Person, Employee, Member, Book, Publisher, Author, and Payment, ensuring data consistency, integrity, and efficient management of library resources and transactions. A Person entity may represent either an Employee or a Member, with the possibility that Employee can also be Member. The system shall store the following attributes for each Person: Person ID, Name (First Name, Middle Name, Last Name), Address, Gender, Date of Birth, and Phone Number(s), where a Person may have multiple associated phone numbers. An Employee must be at least 18 years of age. The Person ID attribute must follow the format “PXXX”, where X denotes a digit between 0 and 9. Each Member is issued a library card, for which details such as Card ID, Issue Date, Membership Level (Silver or Gold), and other relevant information are maintained. The library may also associate Promotions with library cards. Each Promotion is defined by a unique Promotion code and an accompanying description. Each Member is categorized as either Silver or Gold. For Gold Members, a Guest Log is maintained, capturing details such as the Gold Member’s Card ID, Guest ID, Guest Name, Guest Address, and Guest Contact Information. Temporary IDs are assigned to individuals visiting as guests of a Gold Member. Each Guest ID is unique only within the scope of the associated Gold Member and is not globally unique across the entire system. An Employee may belong to one of three categories: Library Supervisor, Cataloging Manager, or Receptionist. The system records the Employment Start Date for each Employee. A Receptionist is required to receive training from a designated Trainer, where a Trainer may be either a Library Supervisor or a Cataloging Manager. Both Library Supervisors and Cataloging Managers are permitted to train multiple Receptionists. Details of Books, including Book ID, Book Title, and other relevant information, are maintained in the system. Books are classified into three categories: Cate. 1, Cate. 2, and Cate. 3. Each Cataloging Manager is responsible for cataloging one category per day, though they may work on different categories on different days. Additionally, a Person

may leave Comments on Books, which include the Comment Time, a Rating Score (ranging from 1 to 5), and the Comment Content. A Publisher may publish multiple Books, whereas each Book is assumed to be published by a single Publisher. The system stores Publisher details, including Publisher ID, Publisher Name, and other relevant information (additional attributes may be defined based on requirements). Similarly, Author details, such as Author ID, Author Name, and other pertinent information, are maintained. A Book may have multiple Authors, and an Author may contribute to multiple Books. A Receptionist is responsible for maintaining records of Borrowing details. Each record includes information about the borrowed Book, Issue Date, Due Date for Return, the Person borrowing the book, the responsible Receptionist, and associated Payment Details. Borrowing records are created only when a Person borrows a Book. Payment Details include Payment ID, Payment Method (e.g., Cash, Debit/Credit Card), Payment Time, and Amount Paid. In addition, the Receptionist is responsible for managing Member Inquiries. Each inquiry is assigned a unique Inquiry ID and records details such as the Inquiry Time and Resolution Status. To monitor member satisfaction and engagement, Members may provide a Rating (ranging from 1 to 5) for the inquiries they have submitted.

## **Project Questions**

### **1. Is the ability to model superclass/subclass relationships likely to be important in the library management system like above? Why or why not?**

In this system, not everyone is just under a natural PERSON class. The description makes it clear that people take on different roles. For instance, some are members, some are employees, and some can be both. Employees also break down even further into supervisors, cataloging managers, and receptionists, each with very different responsibilities. If you think about it, all of these roles still share certain basic information like their name, address, date of birth, phone numbers, and so on. But once you go beyond that, each role starts to need its own extra details. For example, only employees have a start date, only members have library cards, only gold members have guests, and only receptionists handle inquiries and borrowing. Trying to fit all of this into one table would be uncomfortable and would lead to fields that apply to some people but not others. Modeling a Person as the main entity and then creating subcategories for Members and Employees simply reflects the reality of the system. It lets us store the common things once and then add what's unique only where it's needed. It also captures overlapping roles naturally, since one person might wear more than one "hat." Overall, this makes the structure cleaner and easier to maintain.

### **2. Can you think of 5 more rules (other than those explicitly described above) that are likely to be used in the above environment? Please describe how your design would be changed to satisfy your additional rules?**

#### **Rule 1. Borrowing limits based on membership level**

It's reasonable for the library to place limits on how many books a member can have checked out at the same time. Silver members might have a lower limit, while gold members may be allowed more. To

support this, the database would need a place to store the borrowing limit for each membership type, and the borrowing process would need to check the member's current activity before issuing another book. This adds a small amount of structure but reflects how memberships typically work in real libraries.

### **Rule 2. Tracking individual physical copies of a book**

The system currently treats a book as one item, but libraries almost always keep several copies of the same title. To handle this properly, the design would need to separate the idea of a book title from the individual copies on the shelves. Each copy would need its own identifier and status such as available, borrowed, lost, or under repair. Borrowing records would then refer to the specific copy rather than just the title.

### **Rule 3. Overdue fines for late returns**

While the description talks about borrowings and payments, it doesn't mention what happens when a book is returned late. Most libraries charge overdue fines, which means the system would need to record the return date, compare it with the due date, and calculate the amount owed. This would require an additional field or two in the borrowing or payment tables, and possibly a default fine rate that the system can use to compute the charge.

### **Rule 4. Only one active promotion per member at a time**

Promotions are mentioned in the project description, but nothing prevents a member from having several active promotions simultaneously. In reality, libraries tend to enforce one promotion at a time to avoid conflicts or stacking benefits. To support this, the system would either need a way to mark which promotion is currently active or enforce through validation that new promotions don't overlap with existing ones. This adds clarity and prevents contradictory states.

### **Rule 5. Inquiry resolution within a defined time window**

Receptionists handle member inquiries, and the system tracks when an inquiry is created and

whether it has been resolved. A natural rule for improving service quality is requiring inquiries to be resolved within a certain number of hours. If the system stores both the inquiry time and the resolution time, it becomes possible to determine whether the receptionist met the expected timeframe. This would help with reporting and could tie into performance metrics later on.

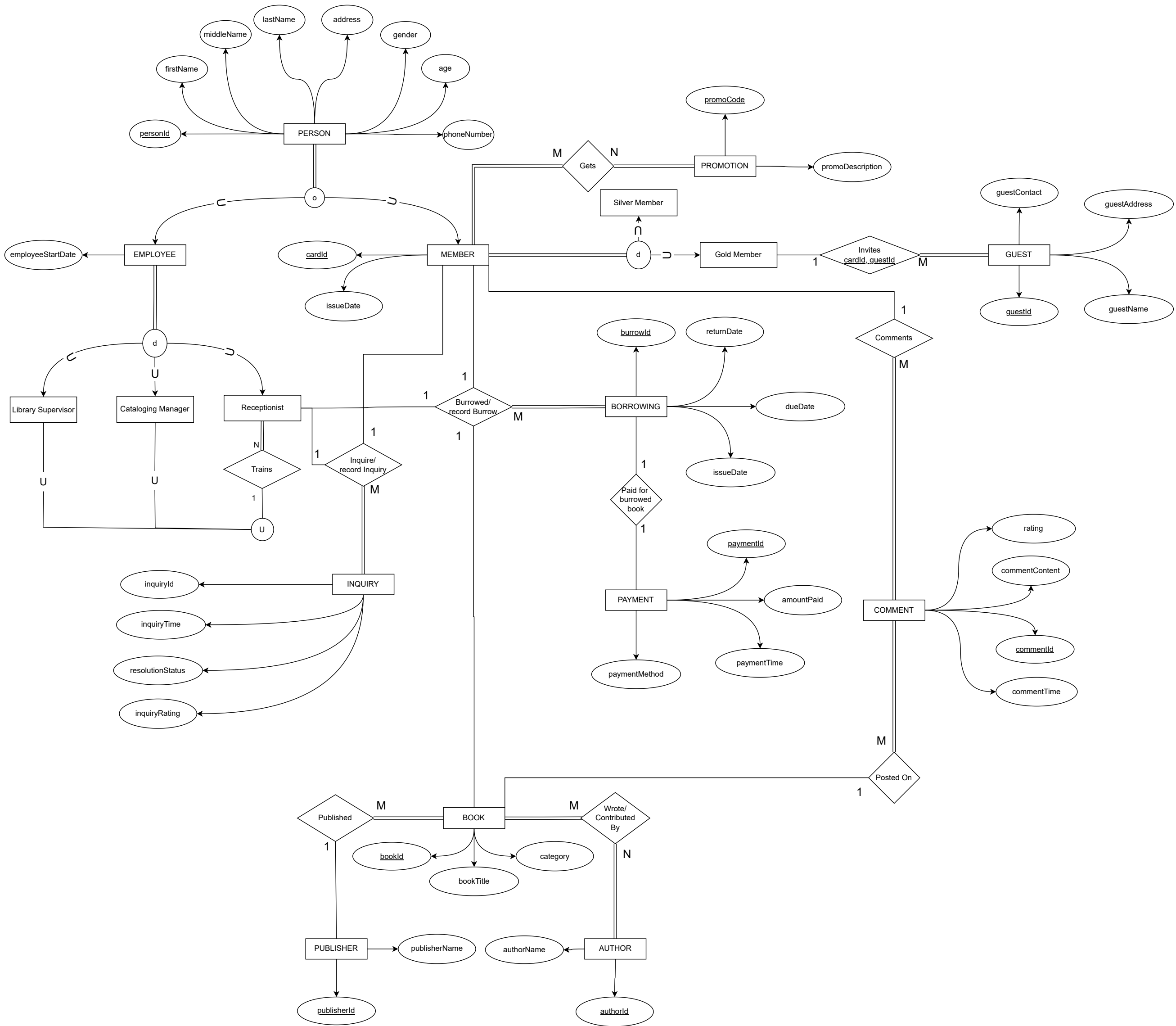
**3. Justify the use of a Relational DBMS like Oracle for this project (Successfully design a relational database system and show all implementation in the final report at Phase IV).**

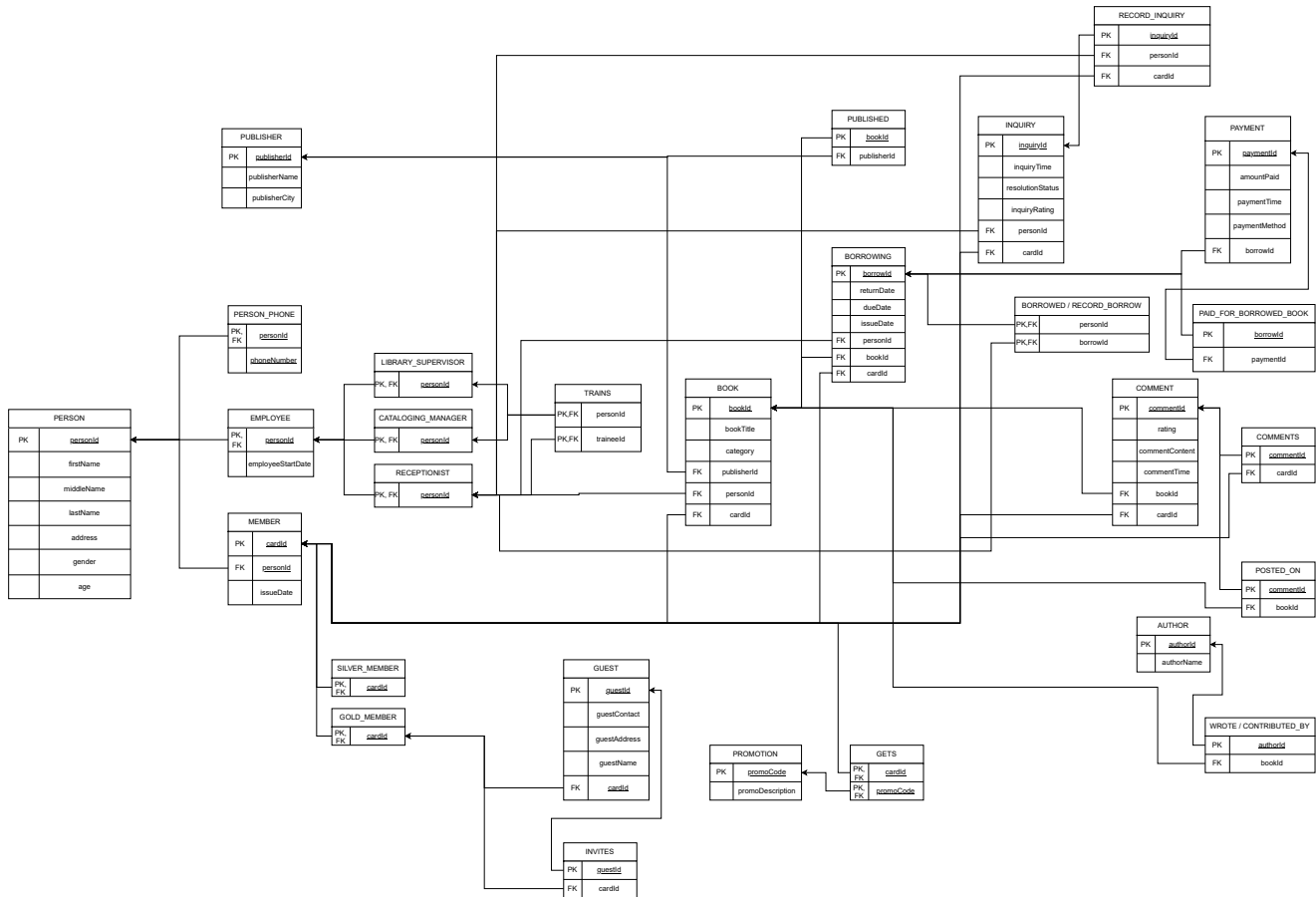
A relational database like Oracle is perfect for this project because the library system is fundamentally built on structure and rules. Almost every action involves relationships. In our case, we have a member borrowing a specific book, an employee logging an inquiry, or a payment clearing a fine. A relational database does well at enforcing these connections using keys, making sure the data stays consistent and traceable. It also guarantees reliability. For instance, if a staff member checks a book in, records a payment, and updates the member's borrowing limit all at once, the system uses transactions to ensure all those steps happen reliably, or none of them do. It focuses on structured relationships, rules enforcement, and transaction reliability, which is why an RDBMS is the ideal foundation for managing the library's complex operations and growth.

## Phase 1

### Assumptions:

1. EMPLOYEE must be at least 18 years old, verified using dateOfBirth from PERSON.
2. personId in PERSON follows the format PXXX.
3. phoneNumber in PERSON is multivalued and stored without sub-attributes.
4. Only MEMBER entities (not Guest) can create BORROWING records or COMMENT entries.
5. Guest is a weak entity of Gold Member, identified by the composite key (cardId, guestId).
6. Every BOOK must participate in Published with exactly one PUBLISHER.
7. BOOK and AUTHOR participate in Wrote/Contributed By as a many-to-many relationship with partial participation.
8. Each BORROWING must include exactly one BOOK, one MEMBER, one Receptionist, and one PAYMENT.
9. PAYMENT has a one-to-one relationship with BORROWING.
10. COMMENT.rating and INQUIRY.inquiryRating must be integers in the range 1–5.
11. Each Receptionist must participate in TrainedBy with either a Library Supervisor or Cataloging Manager.
12. PROMOTION relates to MEMBER through Gets as a many-to-many relationship





## Relational Data Model: DB Project Phase 2

### Important Notes:-

- PK = Primary Key
- FK = Foreign Key
- If a table in diagram has multiple PK in front of columns, then they together form the PK. Example:

|    |         |
|----|---------|
| PK | Column1 |
| PK | Column2 |

Means PK(Column1, Column2)

- All related keys among tables/entities are marked using the arrows in given relational data model

### Primary and Foreign Keys:-

- **personId** is the primary key of PERSON TABLE.
- **personId** acts as the primary key in PERSON\_PHONE but is also a Foreign Key
- **personId** acts as the primary key in EMPLOYEE but is also a Foreign Key
- **cardId** is the Primary Key in MEMBER and **personId** is the Foreign Key
- **personId** acts as the primary key in LIBRARY\_SUPERVISOR but is also a Foreign Key
- **personId** acts as the primary key in CATALOGING\_MANAGER but is also a Foreign Key
- **personId** acts as the primary key in RECEPTIONIST but is also a Foreign Key
- **personId, traineeId** together acts as primary key in TRAINS but separately are Foreign Keys
- **bookId** is the primary key in BOOKS and **personId, publisherId, cardId** separately are Foreign Keys
- **borrowId** is the primary key in BORROWING and **personId, bookId, cardId** separately are Foreign Keys
- **inquiryId** is the primary key in INQUIRY and **personId, cardId** separately are Foreign Keys
- **inquiryId** is the primary key in RECORD\_INQUIRY and **personId, cardId** separately are Foreign Keys
- **publisherId** is the primary key in PUBLISHER

- **bookId** is the primary key in PUBLISHED and **publisherId** is the Foreign Key
- **personId**, **borrowId** together acts as primary key in BORROWED/RECORD\_BORROW but separately are Foreign Keys
- **paymentId** is the primary key in PAYMENT and **borrowId** is the Foreign Key
- **borrowId** is the primary key in PAID\_FOR\_BORROWED\_BOOK and **paymentId** is the Foreign Key
- **commentId** is the primary key in COMMENT and **bookId**, **cardId** separately are Foreign Keys
- **commentId** is the primary key in COMMENTS and **cardId** is the Foreign Key
- **commentId** is the primary key in POSTED\_ON and **bookId** is the Foreign Key
- **authorId** is the primary key in AUTHOR
- **authorId** is the primary key in WROTE/CONTRIBUTED\_BY and **bookId** is the Foreign Key
- **promoCode** is the primary key in PROMOTION
- **cardId**, **promoCode** together acts as primary key in GETS but separately are Foreign Keys
- **guestId** is the primary key in GUEST and **cardId** is the Foreign Key
- **guestId** is the primary key in INVITES and **cardId** is the Foreign Key
- **cardId** acts as the primary key in SILVER\_MEMBER but is also a Foreign Key
- **cardId** acts as the primary key in GOLD\_MEMBER but is also a Foreign Key

CS 6360.002 : DB PROJECT PHASE 3

GROUP : 5

A & B) Normalized tables with their Functional Dependencies:

→ Most of the schema from previous phase, is already in 3NF, because it was derived from a proper EER diagram.

PERSON

|                       |            |             |           |               |
|-----------------------|------------|-------------|-----------|---------------|
| <u>person_id</u> (PK) | first_name | middle_name | last_name | date_of_birth |
|-----------------------|------------|-------------|-----------|---------------|

|        |         |
|--------|---------|
| gender | address |
|--------|---------|

FDs: person\_id → first\_name, middle\_name, last\_name, date\_of\_birth, gender, address

PERSON\_PHONE

|                            |                          |
|----------------------------|--------------------------|
| <u>person_id</u> (PK) (FK) | <u>phone_number</u> (PK) |
|----------------------------|--------------------------|

FDs: (person\_id, phone\_number) → No other attributes

EMPLOYEE

|                            |                     |               |
|----------------------------|---------------------|---------------|
| <u>person_id</u> (PK) (FK) | employee_start_date | employee_type |
|----------------------------|---------------------|---------------|

FDs: person\_id → employee\_start\_date, employee\_type

### LIBRARY\_SUPERVISOR

|                            |
|----------------------------|
| <u>person_id</u> (PK) (FK) |
|----------------------------|

FDs: person\_id  $\rightarrow$  (role membership)

### TRAINING

|                             |                             |                           |
|-----------------------------|-----------------------------|---------------------------|
| <u>trainer_id</u> (PK) (FK) | <u>trainee_id</u> (PK) (FK) | <u>training_date</u> (PK) |
|-----------------------------|-----------------------------|---------------------------|

FDs: (trainer\_id, trainee\_id, training\_date)  $\rightarrow$  No non-key attributes

### MEMBER

|                     |                |            |                  |
|---------------------|----------------|------------|------------------|
| <u>card_id</u> (PK) | person_id (FK) | issue_date | membership_level |
|---------------------|----------------|------------|------------------|

FDs: card\_id  $\rightarrow$  person\_id, issue\_date, membership\_level

### GOLD\_MEMBER

|                          |
|--------------------------|
| <u>card_id</u> (PK) (FK) |
|--------------------------|

FDs: card\_id  $\rightarrow$  (is-gold flag)

### SILVER\_MEMBER

|                          |
|--------------------------|
| <u>card_id</u> (PK) (FK) |
|--------------------------|

FDs: card\_id  $\rightarrow$  (is-silver flag)

### PROMOTION

|                        |                   |
|------------------------|-------------------|
| <u>promo_code</u> (PK) | promo_description |
|------------------------|-------------------|

FDs: promo\_code  $\rightarrow$  promo\_description

### GUEST

|                          |                      |            |               |
|--------------------------|----------------------|------------|---------------|
| <u>card_id</u> (PK) (FK) | <u>guest_id</u> (PK) | guest_name | guest_contact |
|--------------------------|----------------------|------------|---------------|

guest\_address

FDs: (card\_id, guest\_id) → guest\_name, guest\_address, guest\_contact

### MEMBER\_PROMOTION

|                          |                             |
|--------------------------|-----------------------------|
| <u>card_id</u> (PK) (FK) | <u>promo_code</u> (PK) (FK) |
|--------------------------|-----------------------------|

FDs: (card\_id, promo\_code) → No other member

### PUBLISHER

|                          |                |                |
|--------------------------|----------------|----------------|
| <u>publisher_id</u> (PK) | publisher_name | publisher_info |
|--------------------------|----------------|----------------|

FDs: publisher\_id → publisher\_name, publisher\_info

### AUTHOR

|                       |             |             |
|-----------------------|-------------|-------------|
| <u>author_id</u> (PK) | author_name | author_info |
|-----------------------|-------------|-------------|

FDs: author\_id → author\_name, author\_info

### BOOK

|                     |       |          |                   |
|---------------------|-------|----------|-------------------|
| <u>book_id</u> (PK) | title | category | publisher_id (FK) |
|---------------------|-------|----------|-------------------|

publication\_year

FDs: book\_id → title, category, publisher\_id, publication\_year

### BOOK - AUTHOR

|                          |                            |
|--------------------------|----------------------------|
| <u>book_id</u> (PK) (FK) | <u>author_id</u> (PK) (FK) |
|--------------------------|----------------------------|

FDs: (book\_id, author\_id) → No other attribute

### BOOK - COMMENT

|                        |                 |              |        |
|------------------------|-----------------|--------------|--------|
| <u>comment_id</u> (PK) | person_id (FK)  | book_id (FK) | rating |
| comment_time           | comment_content |              |        |

FDs: comment\_id → person\_id, book\_id, rating, comment\_time, comment\_content

### BORROWING

|                       |                             |                         |
|-----------------------|-----------------------------|-------------------------|
| <u>borrow_id</u> (PK) | book_id (FK)                | borrower_person_id (FK) |
| borrower_card_id (FK) | receptionist_person_id (FK) | issue_date              |
| due_date              | return_date                 |                         |

FDs:

borrow\_id → book\_id, borrower\_person\_id, borrower\_card\_id, receptionist\_person\_id, issue\_date, due\_date, return\_date

### RECEPTIONIST

|                            |
|----------------------------|
| <u>person_id</u> (PK) (FK) |
|----------------------------|

FDs: person\_id → (role membership)

### CATALOG - MANAGER

|                            |
|----------------------------|
| <u>person_id</u> (PK) (FK) |
|----------------------------|

FDs: person\_id  $\rightarrow$  (role membership)

### PAYMENT

|                        |                       |                |
|------------------------|-----------------------|----------------|
| <u>payment_id</u> (PK) | borrow_id (UNIQUE FK) | payment_method |
| payment_time           | amount_paid           |                |

FDs: payment\_id  $\rightarrow$  borrow\_id, payment\_method, amount\_paid, payment\_time

### INQUIRY

|                             |                     |                |              |
|-----------------------------|---------------------|----------------|--------------|
| <u>inquiry_id</u> (PK)      | member_card_id (FK) | person_id (FK) | inquiry_time |
| receptionist_person_id (FK) | resolution_status   | inquiry_rating |              |

FDs: inquiry\_id  $\rightarrow$  member\_card\_id, person\_id, inquiry\_time, receptionist\_person\_id, resolution\_status, inquiry\_rating

### CATALOG - ASSIGNMENT

|                         |                             |          |
|-------------------------|-----------------------------|----------|
| <u>assign_date</u> (PK) | manager_person_id (PK) (FK) | category |
|-------------------------|-----------------------------|----------|

FDs: (assign\_date, manager\_person\_id)  $\rightarrow$  category

## **SQL statements to create relational database.**

### **1. PERSON and Phones**

```
CREATE TABLE PERSON (  
    person_id INT PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    middle_name VARCHAR(100),  
    last_name VARCHAR(100) NOT NULL,  
    date_of_birth DATE DEFAULT (CURRENT_DATE) NOT NULL,  
    gender VARCHAR(10),  
    address VARCHAR(400)  
);  
  
CREATE TABLE PERSON_PHONE (  
    person_id INT NOT NULL,  
    phone_number VARCHAR(20) NOT NULL,  
    CONSTRAINT pk_person_phone PRIMARY KEY (person_id, phone_number),  
    CONSTRAINT fk_personphone_person FOREIGN KEY (person_id) REFERENCES  
    PERSON(person_id) ON DELETE CASCADE  
);
```

### **2. EMPLOYEE & ROLE tables**

```
CREATE TABLE EMPLOYEE (  
    person_id INT PRIMARY KEY,  
    employee_start_date DATE DEFAULT (CURRENT_DATE) NOT NULL,  
    employee_type VARCHAR(30) NOT NULL,  
    CONSTRAINT fk_employee_person FOREIGN KEY (person_id) REFERENCES  
    PERSON(person_id) ON DELETE CASCADE,
```

```
CONSTRAINT chk_employee_type CHECK (employee_type IN  
( 'LIBRARY_SUPERVISOR', 'CATALOGING_MANAGER', 'RECEPTIONIST' ))
```

```
);
```

```
CREATE TABLE LIBRARY_SUPERVISOR (
```

```
    person_id INT PRIMARY KEY,
```

```
    CONSTRAINT fk_ls_emp FOREIGN KEY (person_id) REFERENCES  
EMPLOYEE(person_id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE CATALOGING_MANAGER (
```

```
    person_id INT PRIMARY KEY,
```

```
    CONSTRAINT fk_cm_emp FOREIGN KEY (person_id) REFERENCES  
EMPLOYEE(person_id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE RECEPTIONIST (
```

```
    person_id INT PRIMARY KEY,
```

```
    CONSTRAINT fk_r_emp FOREIGN KEY (person_id) REFERENCES EMPLOYEE(person_id)  
ON DELETE CASCADE
```

```
);
```

### **3. TRAINING**

```
CREATE TABLE TRAINING (
```

```
    trainer_id INT NOT NULL,
```

```
    trainee_id INT NOT NULL,
```

```
    training_date DATE DEFAULT (CURDATE()),
```

```
    CONSTRAINT pk_training PRIMARY KEY (trainer_id, trainee_id, training_date),
```

```
    CONSTRAINT fk_training_trainer FOREIGN KEY (trainer_id) REFERENCES  
EMPLOYEE(person_id),
```

```
    CONSTRAINT fk_training_trainee FOREIGN KEY (trainee_id) REFERENCES  
EMPLOYEE(person_id)
```

);

#### **4. MEMBER & GOLD/SILVER & GUEST**

CREATE TABLE MEMBER (

card\_id INT PRIMARY KEY,

person\_id INT NOT NULL,

issue\_date DATE DEFAULT (CURRENT\_DATE) NOT NULL,

membership\_level VARCHAR(10) NOT NULL,

CONSTRAINT fk\_member\_person FOREIGN KEY (person\_id) REFERENCES  
PERSON(person\_id) ON DELETE CASCADE,

CONSTRAINT chk\_member\_level CHECK (membership\_level IN ('SILVER','GOLD'))

);

CREATE TABLE GOLD\_MEMBER (

card\_id INT PRIMARY KEY,

CONSTRAINT fk\_gold\_member FOREIGN KEY (card\_id) REFERENCES MEMBER(card\_id)  
ON DELETE CASCADE

);

CREATE TABLE SILVER\_MEMBER (

card\_id INT PRIMARY KEY,

CONSTRAINT fk\_silver\_member FOREIGN KEY (card\_id) REFERENCES  
MEMBER(card\_id) ON DELETE CASCADE

);

CREATE TABLE GUEST (

card\_id INT NOT NULL,

guest\_id INT NOT NULL,

```

    guest_name VARCHAR(200),
    guest_address VARCHAR(400),
    guest_contact VARCHAR(50),
    CONSTRAINT pk_guest PRIMARY KEY (card_id, guest_id),
    CONSTRAINT fk_guest_member FOREIGN KEY (card_id) REFERENCES
    MEMBER(card_id) ON DELETE CASCADE
);

```

## 5. PROMOTION

```

CREATE TABLE PROMOTION (
    promo_code VARCHAR(20) PRIMARY KEY,
    promo_description VARCHAR(400)
);

```

```

CREATE TABLE MEMBER_PROMOTION (
    card_id INT NOT NULL,
    promo_code VARCHAR(20) NOT NULL,
    CONSTRAINT pk_member_promo PRIMARY KEY (card_id, promo_code),
    CONSTRAINT fk_mpromo_card FOREIGN KEY (card_id) REFERENCES MEMBER(card_id)
    ON DELETE CASCADE,
    CONSTRAINT fk_mpromo_promo FOREIGN KEY (promo_code) REFERENCES
    PROMOTION(promo_code)
);

```

## 6. PUBLISHER, AUTHOR, BOOK, BOOK\_AUTHOR

```

CREATE TABLE PUBLISHER (
    publisher_id INT PRIMARY KEY,
    publisher_name VARCHAR(200) NOT NULL,

```

```
publisher_info VARCHAR(400)
);
```

```
CREATE TABLE AUTHOR (
    author_id INT PRIMARY KEY,
    author_name VARCHAR(200) NOT NULL,
    author_info VARCHAR(400)
);
```

```
CREATE TABLE BOOK (
    book_id INT PRIMARY KEY,
    title VARCHAR(400) NOT NULL,
    category VARCHAR(10) NOT NULL,
    publisher_id INT,
    publication_year INT,
    CONSTRAINT fk_book_publisher FOREIGN KEY (publisher_id) REFERENCES
PUBLISHER(publisher_id),
    CONSTRAINT chk_book_category CHECK (category IN ('Cate1','Cate2','Cate3'))
);
```

```
CREATE TABLE BOOK_AUTHOR (
    book_id INT NOT NULL,
    author_id INT NOT NULL,
    CONSTRAINT pk_book_author PRIMARY KEY (book_id, author_id),
    CONSTRAINT fk_ba_book FOREIGN KEY (book_id) REFERENCES BOOK(book_id) ON
DELETE CASCADE,
    CONSTRAINT fk_ba_author FOREIGN KEY (author_id) REFERENCES AUTHOR(author_id)
ON DELETE CASCADE
);
```

## **7. COMMENT**

```
CREATE TABLE BOOK_COMMENT (  
    comment_id INT PRIMARY KEY,  
    person_id INT NOT NULL,  
    book_id INT NOT NULL,  
    comment_time DATE DEFAULT (CURDATE()),  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comment_content VARCHAR(2000),  
    CONSTRAINT fk_comment_person FOREIGN KEY (person_id) REFERENCES  
PERSON(person_id),  
    CONSTRAINT fk_comment_book FOREIGN KEY (book_id) REFERENCES BOOK(book_id)  
);
```

## **8. BORROWING & PAYMENT**

```
CREATE TABLE BORROWING (  
    borrow_id INT PRIMARY KEY,  
    book_id INT NOT NULL,  
    borrower_person_id INT NOT NULL,  
    borrower_card_id INT,  
    receptionist_person_id INT NOT NULL,  
    issue_date DATE DEFAULT (CURRENT_DATE) NOT NULL,  
    due_date DATE,  
    return_date DATE,  
    CONSTRAINT fk_borrow_book FOREIGN KEY (book_id) REFERENCES BOOK(book_id),  
    CONSTRAINT fk_borrow_borrower FOREIGN KEY (borrower_person_id) REFERENCES  
PERSON(person_id),  
    CONSTRAINT fk_borrow_recep FOREIGN KEY (receptionist_person_id) REFERENCES  
RECEPTIONIST(person_id),  
    CONSTRAINT fk_borrow_card FOREIGN KEY (borrower_card_id) REFERENCES  
MEMBER(card_id)
```

);

CREATE TABLE PAYMENT (

payment\_id INT PRIMARY KEY,

borrow\_id INT UNIQUE,

payment\_method VARCHAR(30),

payment\_time DATE DEFAULT (CURDATE()),

amount\_paid DECIMAL(10,2) DEFAULT 0,

CONSTRAINT fk\_payment\_borrow FOREIGN KEY (borrow\_id) REFERENCES  
BORROWING(borrow\_id) ON DELETE SET NULL

);

## 9. INQUIRY

CREATE TABLE INQUIRY (

inquiry\_id INT PRIMARY KEY,

member\_card\_id INT NOT NULL,

person\_id INT,

receptionist\_person\_id INT,

inquiry\_time DATE DEFAULT (CURDATE()),

resolution\_status VARCHAR(50),

inquiry\_rating INT CHECK (inquiry\_rating BETWEEN 1 AND 5),

CONSTRAINT fk\_inquiry\_member FOREIGN KEY (member\_card\_id) REFERENCES  
MEMBER(card\_id),

CONSTRAINT fk\_inquiry\_person FOREIGN KEY (person\_id) REFERENCES  
PERSON(person\_id),

CONSTRAINT fk\_inquiry\_recep FOREIGN KEY (receptionist\_person\_id) REFERENCES  
RECEPTIONIST(person\_id)

);

## 10. CATALOGING\_ASSIGNMENT

```

CREATE TABLE CATALOGING_ASSIGNMENT (
    assign_date DATE DEFAULT (CURRENT_DATE) NOT NULL,
    manager_person_id INT NOT NULL,
    category VARCHAR(10) NOT NULL,
    CONSTRAINT pk_catalog_assign PRIMARY KEY (assign_date, manager_person_id),
    CONSTRAINT fk_catalog_manager FOREIGN KEY (manager_person_id) REFERENCES
    CATALOGING_MANAGER(person_id),
    CONSTRAINT chk_catalog_cat CHECK (category IN ('Cate1','Cate2','Cate3'))
);

```

## TRIGGERS:

### 1. Enforce EMPLOYEE age >= 18 at insertion/update:

```

DELIMITER $$

CREATE TRIGGER trg_employee_age_check_update
BEFORE UPDATE ON EMPLOYEE
FOR EACH ROW
BEGIN
    DECLARE dob DATE;
    DECLARE age_years INT;

    SELECT date_of_birth INTO dob FROM PERSON WHERE person_id = NEW.person_id;
    SET age_years = FLOOR(TIMESTAMPDIFF(MONTH, dob, CURDATE()) / 12);

    IF age_years < 18 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee must be at least 18 years old.';
    END IF;

```

END\$\$

DELIMITER ;

2. **Enforce TRAINING roles: trainer must be SUPERVISOR or CATALOGING\_MANAGER; trainee must be RECEPTIONIST.**

DELIMITER \$\$

CREATE TRIGGER trg\_training\_role\_check\_update

BEFORE UPDATE ON TRAINING

FOR EACH ROW

BEGIN

DECLARE trainer\_type VARCHAR(30);

DECLARE trainee\_type VARCHAR(30);

SELECT employee\_type INTO trainer\_type FROM EMPLOYEE WHERE person\_id = NEW.trainer\_id;

SELECT employee\_type INTO trainee\_type FROM EMPLOYEE WHERE person\_id = NEW.trainee\_id;

IF trainer\_type NOT IN ('LIBRARY\_SUPERVISOR','CATALOGING\_MANAGER') THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE\_TEXT = 'Trainer must be Library Supervisor or Cataloging Manager.';

END IF;

IF trainee\_type <> 'RECEPTIONIST' THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE\_TEXT = 'Trainee must be a Receptionist.';

END IF;

END\$\$

DELIMITER ;

3. **If BORROWING.borrower\_card\_id provided, ensure that MEMBER.card\_id belongs to borrower\_person\_id (consistency of which person owns the card):**

DELIMITER \$\$

```

CREATE TRIGGER trg_borrow_card_consistency_update
BEFORE UPDATE ON BORROWING
FOR EACH ROW
BEGIN
    DECLARE owner_person INT;

    IF NEW.borrower_card_id IS NOT NULL THEN

        SELECT person_id INTO owner_person FROM MEMBER WHERE card_id =
NEW.borrower_card_id;

        IF owner_person <> NEW.borrower_person_id THEN

            SIGNAL SQLSTATE '45000'

            SET MESSAGE_TEXT = 'Provided borrower_card_id does not belong to
borrower_person_id.';

        END IF;

    END IF;

END$$

DELIMITER ;

```

#### **D. SQL statements to create views.**

- 1. TopGoldMember — First Name, Last Name, Date of membership enrollment of Gold members who borrowed > 5 books in the past month:**

```

CREATE OR REPLACE VIEW TopGoldMember AS

SELECT p.first_name, p.last_name, m.issue_date

FROM MEMBER m

JOIN PERSON p ON m.person_id = p.person_id

JOIN (

    SELECT borrower_person_id, COUNT(*) AS borrow_count

    FROM BORROWING

    WHERE issue_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)

    GROUP BY borrower_person_id

```

```
) b ON b.borrower_person_id = m.person_id
```

```
WHERE m.membership_level = 'GOLD'
```

```
AND b.borrow_count > 5;
```

- 2. PopularBooks — details of the most borrowed books over the past year. I interpret "most borrowed books" as book(s) with maximum borrow count in last 12 months.**

```
CREATE OR REPLACE VIEW PopularBooks AS
```

```
WITH borrow_counts AS (
```

```
    SELECT book_id, COUNT(*) AS cnt
```

```
    FROM BORROWING
```

```
    WHERE issue_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
```

```
    GROUP BY book_id
```

```
),
```

```
max_count AS (
```

```
    SELECT MAX(cnt) AS max_cnt FROM borrow_counts
```

```
)
```

```
SELECT bk.book_id, bk.title, bk.category, bk.publisher_id, bc.cnt AS borrow_count
```

```
FROM BOOK bk
```

```
JOIN borrow_counts bc ON bk.book_id = bc.book_id
```

```
CROSS JOIN max_count
```

```
WHERE bc.cnt = max_count.max_cnt;
```

- 3. BestRatingPublisher — publishers whose books all have at least a 4.0 average rating. Implementation: compute average rating per book, then for each publisher ensure MIN(avg\_rating\_of\_its\_books) >= 4.0 (i.e., all books have avg >= 4).**

```
CREATE OR REPLACE VIEW BestRatingPublisher AS
```

```
WITH book_avg AS (
```

```
    SELECT book_id, AVG(rating) AS avg_rating
```

```
    FROM BOOK_COMMENT
```

```

GROUP BY book_id
)
SELECT p.publisher_name
FROM PUBLISHER p
JOIN BOOK b ON b.publisher_id = p.publisher_id
LEFT JOIN book_avg ba ON ba.book_id = b.book_id
GROUP BY p.publisher_name
HAVING MIN(ba.avg_rating) >= 4.0;

```

**4. PotentialGoldMember - Silver members who borrowed books every month in the past year. We interpret "every month in the past year" as 12 distinct month buckets in last 12 months.**

```

CREATE OR REPLACE VIEW PotentialGoldMember AS
WITH months_borrowed AS (
    SELECT m.card_id, DATE_FORMAT(b.issue_date, '%Y-%m-01') AS month_start
    FROM MEMBER m
    JOIN BORROWING b ON b.borrower_person_id = m.person_id
    WHERE b.issue_date >= DATE_SUB(DATE_FORMAT(CURDATE(), '%Y-%m-01'),
    INTERVAL 11 MONTH)
    AND m.membership_level = 'SILVER'
    GROUP BY m.card_id, DATE_FORMAT(b.issue_date, '%Y-%m-01')
),
monthly_counts AS (
    SELECT card_id, COUNT(DISTINCT month_start) AS months_with_borrow
    FROM months_borrowed
    GROUP BY card_id
)
SELECT p.first_name, p.last_name, ph.phone_number, m.card_id

```

```
FROM MEMBER m
JOIN PERSON p ON m.person_id = p.person_id
LEFT JOIN PERSON_PHONE ph ON ph.person_id = p.person_id
JOIN monthly_counts mc ON mc.card_id = m.card_id
WHERE m.membership_level = 'SILVER'
AND mc.months_with_borrow = 12;
```

**5. ActiveReceptionist — receptionists who resolved more than 5 inquiries from members in the past month.**

```
CREATE OR REPLACE VIEW ActiveReceptionist AS
SELECT p.person_id, p.first_name, p.last_name, COUNT(*) AS resolved_count
FROM INQUIRY i
JOIN RECEPTIONIST r ON i.receptionist_person_id = r.person_id
JOIN PERSON p ON p.person_id = r.person_id
WHERE i.resolution_status = 'Resolved'
AND i.inquiry_time >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY p.person_id, p.first_name, p.last_name
HAVING COUNT(*) > 5;
```

**E. SQL statements to run the following queries.**

**1. List details of all the supervisors of the library hired in the past two months.**

```
SELECT p.person_id, p.first_name, p.last_name, e.employee_start_date, p.date_of_birth,
p.address, p.gender
FROM EMPLOYEE e
JOIN LIBRARY_SUPERVISOR ls ON ls.person_id = e.person_id
JOIN PERSON p ON p.person_id = e.person_id
WHERE e.employee_start_date >= DATE_SUB(CURDATE(), INTERVAL 2 MONTH);
```

**2. Find the names of employees who are also members and the books they have borrowed in the past month.**

```
SELECT emp.person_id, p.first_name, p.last_name, bk.book_id, bk.title, br.issue_date
FROM EMPLOYEE emp
JOIN PERSON p ON p.person_id = emp.person_id
JOIN MEMBER m ON m.person_id = emp.person_id
JOIN BORROWING br ON br.borrower_person_id = emp.person_id
JOIN BOOK bk ON bk.book_id = br.book_id
WHERE br.issue_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH);
```

**3. Find the average number of books borrowed by the top five gold members in the library.**

```
WITH gold_borrows AS (
    SELECT m.card_id, m.person_id, COUNT(*) AS total_borrows
    FROM MEMBER m
    JOIN BORROWING b ON b.borrower_person_id = m.person_id
    WHERE m.membership_level = 'GOLD'
    GROUP BY m.card_id, m.person_id
),
top5 AS (
    SELECT card_id, person_id, total_borrows
    FROM gold_borrows
    ORDER BY total_borrows DESC
    LIMIT 5
)
SELECT AVG(total_borrows) AS avg_borrows_by_top5
FROM top5;
```

**4. Find the name of the publishers and the title of the most popular book for each publisher.**

```
WITH pub_book_counts AS (  
    SELECT p.publisher_id, p.publisher_name, b.book_id, b.title, COUNT(br.borrow_id) AS  
    borrow_count  
  
    FROM PUBLISHER p  
  
    JOIN BOOK b ON b.publisher_id = p.publisher_id  
  
    LEFT JOIN BORROWING br ON br.book_id = b.book_id  
  
    GROUP BY p.publisher_id, p.publisher_name, b.book_id, b.title  
)  
  
max_per_publisher AS (  
    SELECT publisher_id, MAX(borrow_count) AS max_count  
  
    FROM pub_book_counts  
  
    GROUP BY publisher_id  
)  
  
SELECT pbc.publisher_name, pbc.title, pbc.borrow_count  
  
FROM pub_book_counts pbc  
  
JOIN max_per_publisher mpp ON pbc.publisher_id = mpp.publisher_id AND pbc.borrow_count  
= mpp.max_count  
  
ORDER BY pbc.publisher_name;
```

**5. Find names of books that were not borrowed in the last 5 months.**

```
SELECT b.book_id, b.title  
  
FROM BOOK b  
  
LEFT JOIN BORROWING br ON br.book_id = b.book_id  
  
AND br.issue_date >= DATE_SUB(CURDATE(), INTERVAL 5 MONTH)  
  
WHERE br.borrow_id IS NULL;
```

**6. Find the members who have borrowed all the books written by the most popular author.**

```
WITH author_borrow_counts AS (  
    SELECT ba.author_id, a.author_name, SUM(IFNULL(bc.cnt, 0)) AS total_borrows  
    FROM BOOK_AUTHOR ba  
    JOIN AUTHOR a ON a.author_id = ba.author_id  
    LEFT JOIN (  
        SELECT book_id, COUNT(*) AS cnt FROM BORROWING GROUP BY book_id  
    ) bc ON bc.book_id = ba.book_id  
    GROUP BY ba.author_id, a.author_name  
,  
most_pop_author AS (  
    SELECT author_id  
    FROM author_borrow_counts  
    WHERE total_borrows = (SELECT MAX(total_borrows) FROM author_borrow_counts)  
,  
author_books AS (  
    SELECT DISTINCT book_id  
    FROM BOOK_AUTHOR  
    WHERE author_id IN (SELECT author_id FROM most_pop_author)  
,  
author_books_count AS (  
    SELECT COUNT(*) AS cnt FROM author_books  
,  
member_borrowed_count AS (  
    SELECT m.card_id, m.person_id, COUNT(DISTINCT b.book_id) AS borrowed_count  
    FROM MEMBER m
```

```

JOIN BORROWING b ON b.borrower_person_id = m.person_id
WHERE b.book_id IN (SELECT book_id FROM author_books)
GROUP BY m.card_id, m.person_id
)
SELECT p.person_id, p.first_name, p.last_name, m.card_id
FROM member_borrowed_count mb
JOIN PERSON p ON p.person_id = mb.person_id
JOIN MEMBER m ON m.card_id = mb.card_id
WHERE mb.borrowed_count = (SELECT cnt FROM author_books_count)

```

### **7. Find the Gold Member with the greatest number of guests.**

```

WITH author_borrow_counts AS (
    SELECT ba.author_id, a.author_name, SUM(IFNULL(bc.cnt, 0)) AS total_borrows
    FROM BOOK_AUTHOR ba
    JOIN AUTHOR a ON a.author_id = ba.author_id
    LEFT JOIN (
        SELECT book_id, COUNT(*) AS cnt FROM BORROWING GROUP BY book_id
    ) bc ON bc.book_id = ba.book_id
    GROUP BY ba.author_id, a.author_name
),
most_pop_author AS (
    SELECT author_id
    FROM author_borrow_counts
    WHERE total_borrows = (SELECT MAX(total_borrows) FROM author_borrow_counts)
),
author_books AS (
    SELECT DISTINCT book_id

```

```

FROM BOOK_AUTHOR
WHERE author_id IN (SELECT author_id FROM most_pop_author)
),
author_books_count AS (
    SELECT COUNT(*) AS cnt FROM author_books
),
member_borrowed_count AS (
    SELECT m.card_id, m.person_id, COUNT(DISTINCT b.book_id) AS borrowed_count
    FROM MEMBER m
    JOIN BORROWING b ON b.borrower_person_id = m.person_id
    WHERE b.book_id IN (SELECT book_id FROM author_books)
    GROUP BY m.card_id, m.person_id
)
SELECT p.person_id, p.first_name, p.last_name, m.card_id
FROM member_borrowed_count mb
JOIN PERSON p ON p.person_id = mb.person_id
JOIN MEMBER m ON m.card_id = mb.card_id
WHERE mb.borrowed_count = (SELECT cnt FROM author_books_count);

```

**8. Find the year with the maximum number of books borrowed.**

```

SELECT YEAR(issue_date) AS borrow_year, COUNT(*) AS total_borrows
FROM BORROWING
GROUP BY YEAR(issue_date)
ORDER BY total_borrows DESC
LIMIT 1;

```

**9. Find the names of members who borrowed the most popular books.**

```
SELECT DISTINCT p.person_id, p.first_name, p.last_name, m.card_id
FROM MEMBER m
JOIN PERSON p ON p.person_id = m.person_id
JOIN BORROWING b ON b.borrower_person_id = m.person_id
WHERE b.book_id IN (
    SELECT book_id FROM (
        SELECT book_id, COUNT(*) AS cnt
        FROM BORROWING
        WHERE issue_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
        GROUP BY book_id
        HAVING COUNT(*) = (
            SELECT MAX(cnt) FROM (
                SELECT COUNT(*) AS cnt
                FROM BORROWING
                WHERE issue_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
                GROUP BY book_id
            ) AS max_counts
        )
    ) AS popular_books
)
AND b.issue_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH);
```

**10. List all the employees that have enrolled into gold membership within a month of being employed.**

```
SELECT e.person_id, p.first_name, p.last_name, e.employee_start_date, m.issue_date AS
membership_issue_date
FROM EMPLOYEE e
```

```

JOIN MEMBER m ON m.person_id = e.person_id

JOIN PERSON p ON p.person_id = e.person_id

WHERE m.membership_level = 'GOLD'

AND m.issue_date BETWEEN e.employee_start_date AND
DATE_ADD(e.employee_start_date, INTERVAL 30 DAY);

```

**11. Find the names of receptionists with an average rating of 4.0 from the inquiries they resolved.**

```

SELECT r.person_id, p.first_name, p.last_name, AVG(i.inquiry_rating) AS avg_rating
FROM INQUIRY i

JOIN RECEPTIONIST r ON r.person_id = i.receptionist_person_id

JOIN PERSON p ON p.person_id = r.person_id

WHERE i.resolution_status = 'Resolved'

AND i.inquiry_rating IS NOT NULL

GROUP BY r.person_id, p.first_name, p.last_name

HAVING AVG(i.inquiry_rating) >= 4.0;

```

**12. Find the names of receptionists and their trainers who resolve at least 2 inquiries every month in the past 3 months.**

```

WITH RECURSIVE last_three_months AS (

    SELECT DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 0 MONTH), '%Y-%m-01') AS
mon_start

    UNION ALL

    SELECT DATE_FORMAT(DATE_SUB(mon_start, INTERVAL 1 MONTH), '%Y-%m-01')

FROM last_three_months

WHERE DATE_SUB(mon_start, INTERVAL 1 MONTH) >=
DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 2 MONTH), '%Y-%m-01')

),

re_for_month AS (

```

```

SELECT r.person_id AS recep_id, DATE_FORMAT(i.inquiry_time, '%Y-%m-01') AS
mon_start, COUNT(*) AS resolved_count

FROM INQUIRY i

JOIN RECEPTIONIST r ON r.person_id = i.receptionist_person_id

WHERE i.resolution_status = 'Resolved'

AND i.inquiry_time >= DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 2 MONTH),
'%Y-%m-01')

GROUP BY r.person_id, DATE_FORMAT(i.inquiry_time, '%Y-%m-01')
),
recep_months AS (
SELECT recep_id, COUNT(*) AS months_with_2plus
FROM re_for_month
WHERE resolved_count >= 2
GROUP BY recep_id
)
SELECT pr.recep_id AS receptionist_id, per_r.first_name AS recep_first, per_r.last_name AS
recep_last,

t.trainer_id AS trainer_id, per_t.first_name AS trainer_first, per_t.last_name AS
trainer_last

FROM recep_months pr

JOIN RECEPTIONIST rct ON rct.person_id = pr.recep_id

JOIN PERSON per_r ON per_r.person_id = rct.person_id

JOIN TRAINING t ON t.trainee_id = rct.person_id

JOIN PERSON per_t ON per_t.person_id = t.trainer_id

WHERE pr.months_with_2plus = 3;

```

### 13. List the employee who trained the greatest number of receptionists.

```

SELECT t.trainer_id, p.first_name, p.last_name, COUNT(DISTINCT t.trainee_id) AS
num_receptionists_trained

```

```

FROM TRAINING t
JOIN PERSON p ON p.person_id = t.trainer_id
GROUP BY t.trainer_id, p.first_name, p.last_name
HAVING COUNT(DISTINCT t.trainee_id) = (
    SELECT MAX(cnt) FROM (
        SELECT COUNT(DISTINCT trainee_id) AS cnt
        FROM TRAINING
        GROUP BY trainer_id
    ) AS trainer_counts
);

```

**14. List the Cataloging Managers who cataloged all categories every week in the past 4 weeks.**

```

WITH mgr_week_cat AS (
    SELECT ca.manager_person_id,
        YEARWEEK(ca.assign_date, 1) AS wk,
        COUNT(DISTINCT ca.category) AS cat_count
    FROM CATALOGING_ASSIGNMENT ca
    WHERE ca.assign_date >= DATE_SUB(CURDATE(), INTERVAL 21 DAY)
    GROUP BY ca.manager_person_id, YEARWEEK(ca.assign_date, 1)
)
SELECT p.person_id, p.first_name, p.last_name, count(*), Min(cat_count)
FROM mgr_week_cat mw
JOIN CATALOGING_MANAGER cm ON cm.person_id = mw.manager_person_id
JOIN PERSON p ON p.person_id = cm.person_id
GROUP BY p.person_id, p.first_name, p.last_name
HAVING COUNT(*) = 4
    AND MIN(cat_count) = 3;

```

